

Design and Development of a Web-Based Formula One Historical Data and Replay System

Student ID: F538783

Date: 24/08/26

Table of Contents

1 Introduction.....	3
1.1 Motivation.....	3
1.2 Aim and Objectives.....	3
Aim.....	3
Objectives.....	3
1.3 Relevance.....	4
1.4 Report Structure.....	4
2 Background.....	5
2.1 F1 Data.....	5
2.2 Data Sources.....	6
2.3 Requirements.....	7
Historical Data.....	7
Live and Replay.....	7
3 Design.....	8
3.1 Architecture.....	8
3.2 Data Flow.....	9
3.3 UI Design.....	10
Old Design:.....	10
Current Design:.....	11
4 Implementation.....	13
4.1 API Integration.....	13
4.2 Data Processing.....	14

4.3 Frontend.....	16
4.4 Historical Data.....	18
4.5 Replay System.....	19
4.6 Timing Tower.....	21
4.7 Mini Map.....	23
4.8 Performance.....	24
5 Testing.....	25
5.1 Approach.....	25
5.2 Testing Results.....	28
6 Evaluation.....	29
6.1 Objective Evaluation.....	29
Information Hub.....	29
Mini Map.....	29
Timing Tower.....	30
Historic Results.....	30
Mobile Compatibility.....	31
Race Strategy.....	31
6.2 The Positives.....	32
6.3 The Negatives.....	33
6.4 Future Improvements.....	34
7 Conclusion.....	36
Appendix.....	36
Appendix A:.....	36
References:.....	36
AI Disclosure.....	37

1 Introduction

1.1 Motivation

Formula one is a very data driven sport, with data playing an increasing role in both operation of the teams and the experience of the fans. During a race weekend there is a large amount of data that is produced from session results, lap times, penalties, standings and more... While some of this information is accessible to fans there is a varying degree of how the data is accessed and how accessible that data is to view. This is an extremely fine line to avoid crowding on the web page but also to satisfy even the most hardcore fans of the sport.

There are several options available to fans of the sport to view the data that the sport produces and the main source you will find is the official F1 website, provides a wide range of data from lap times and race information being the most accessible option for fans as it is free to access for everyone, with a mobile app that is supplementary to the site. The main issue with this site is the fact it is designed to support a broadcast which means the actual data itself is designed in a way that is supplementary to the live broadcast rather than stand on its own.

Another option to access data as a fan is third-party websites which will provide more information but at times at the cost of accessibility, this at times can lead the site to feel messy but also lack a clear flow and hierarchy in how the site can be used. Others provide a more accessible experience but at the cost of what data is exposed to the users. In my experience accessibility and depth go hand in hand and that is the line I wish to tread to find a middle ground.

This project aims to meet in the middle depth and accessibility to provide an experience that everyone can enjoy with data deep enough for enthusiasts but accessible enough for everyone to enjoy. A particular focus of the project is historical races to be able to go back into the past and replay the races as if they were live presenting the races as events rather than losing them behind the stats and final results.

1.2 Aim and Objectives

Aim

The overall aim for this project is to develop a web-page that presents information of a race weekend in a manner that is accessible to the average fan but deep enough for the enthusiast viewer.

Objectives

- Develop a hub of information for Formula One allowing users to break down a race weekend showing results from each session but also information as a whole like drivers and team standings
- Develop a live mini-map displaying where each driver is on the track at any time

- Implement a timing tower that updates through a session with varying information for Race and other sessions. The timing tower will display relevant information to the session such as position, gap to the next driver, gap to leader, current tyre compound and lap time
- Develop a way for users to go back through history and view past f1 results from previous seasons and view a replay of those sessions
- Develop an interface that is compatible with mobile and PC devices
- Implement race strategy visualisation to display how a race is unfolding with varying strategies in play

1.3 Relevance

The computer science MSc has provided experience from multiple aspects of computer science including Artificial Intelligence, Python and database management with Pandas, Web Development and Software Engineering. For this project I decided to build upon the Web Development module and take what I learned and apply it in a different context correcting past mistakes and learning more complex workflows rather than a mostly static site, creating a dynamic presentation of data from an API

The use of an API was central to the design of the web page and thus it became very important to look through and understand the endpoints to access the data I can access via the API as well as the structure of the data output and how I can manipulate it. Efficiency also quickly became a consideration since every API call takes time to return a response, I had to think about what data I need and the best way to get it with the minimal amount of API calls.

The project also allowed me to take what was learned on the web development module and apply it in a vastly more complex page which provided a steep learning curve which would have been impossible without this baseline knowledge. The three languages used for this module were HTML, JavaScript and CSS since I was already familiar with them but allows me to use them in a completely different way to what I have done previously with the page being far more weighted to the JavaScript running most of the front end.

1.4 Report Structure

This report will be going through the process I underwent for the project from the concept phase all the way through to the final product taking deeper looks at the background of the project and other related works, design and why certain decisions were made, the implementation of the design exploring what went well but also any changes that had to be made and finally the testing and evaluation phase outline how the project went and outline if the objective set out were met.

2 Background

2.1 F1 Data

During a formula one session there is an abundance of data produced and is manipulated in many ways to give insights from the live broadcast to information breakdowns in media to third party information websites and the way it is manipulated matters vastly as the information needed in a practice session is not the same as a race session. F1 data is divided into 4 categories timing data, position data, tyre data and car telemetry data.

Timing data is the is produced during a session as cars go around the track posting lap times, this includes overall lap time, sector times that make up the lap time, gaps to the other cars and the average speed during a lap. Timing data is very useful to fans of the sport as it paints a picture on who is fastest during a session, especially in qualifying sessions where lap time builds the grid so it is very important. During a race fans may be most interested in the gap data since this will show who is making inroads on the car ahead or pulling away from the car behind.

Position data is mostly data that is used after a session showing final results but can be used in live data showing a driver's current position or how many places a driver has made up during a race. As mentioned prior this is also used in session results showing how the event ended up, whether anyone retired from the session/did not take part and how many laps were completed.

Tyre data is where we start getting into more in depth analysis where this shows how many laps a tyre has done in a session, which compound is used currently but if I want to get deeper this is where it can get very deep if you start considering how many laps on average each driver has done on each compounds as well as lap time degradation as tyre wear increases as this will decide and influence each teams strategy in the race. This type of data is also used to show how the strategies are unfolding live showing what compound was used when and how many laps were/still are being done.

Finally, car telemetry data is used primarily for live broadcast/replay showing a driver's data at any given moment specifically related to the car such as current gear, speed, battery percentage all of which is useful when watching something live. This data is not very useful outside of this unless you are tracking all the drivers' top speeds or number of gear shifts in a lap, building a data base with this info. During a live broadcast/replay this information is specifically useful to understand a drivers/teams battery strategy since they deploy at different places on the track especially in 2026 with new rules or getting a driver's top speed when fighting/in practice as this can give an insight on a driver's set-up and can be running less wing in the aerodynamics. The data produced here does produce useful information at face value but a lot of the use is in breakdowns when trying to gain a deeper insight in how a driver is driving.

The relevance of each data type depends on if the session is being viewed live, as a replay or historic as a session result screen. Live sessions require live updates and different data while the session progresses such as positions, timing and telemetry

particularly for features such as a timing tower and mini map. For a replay system this uses historical data that is processed in a way that is perceived by the system as live which would behave in the same way as live and thus the same information is of use to the replay system. For historic data you only need a snapshot of the data after the race and no live updating is necessary and can be processed after a session is finished.

2.2 Data Sources

For collecting data about Formula One for a development application there is no publicly available way to access the data directly from F1 themselves and that is where APIs come in to act as a middle man to give the data to engineers such as myself and use the data to create applications for yourself and fans of the sport. When the data is generated by F1, the API's collect the data that is produced and expose this data to request datasets and via an API call I can get the data via specific endpoints depending on what data I want at a given time. There are 2 major players in this space, OpenF1 and FastF1.

I opted for the OpenF1 API this is for a few major reasons. Firstly, I decided as my first decision on if this project will be a local application or a web-app as reasoned in my design document introduction ([Appendix A](#)) which was to be a web-app and as such I needed the API to work in JavaScript which FastF1 does not support and only works in python. Secondly, OpenF1 is better suited for live streaming data. Since the library is designed for web applications the live data streaming is more robust, Fast F1 supports it but its intended purpose is for python data analysis. Where Fast F1 is better and could be integrated at a later date is its access to past data is a lot wider, OpenF1 only supports the 2023 season and onwards whereas FastF1 is a lot deeper with data ranging further back. If this was a local based application in python Fast F1 is the no brainer but for this specific project OpenF1 was the obvious choice.

The structure of the API is very intuitive and behaves a lot like an SQL database for historic data via the REST API with a session key behaving like a super key to access all endpoints with different data within the endpoints. To make an API call the default URL is "https://api.openf1.org/v1/" and all the endpoints are added onto the end of the URL, for example the car data would be "https://api.openf1.org/v1/car_data" which would include all the data for a cars speed, gears and more. For a timing tower for example you would make use of the position, drivers, intervals and if you want to show how many places a driver has gained or lost you would need the starting grid endpoint. Since historic data used the REST API this means every time you make an API call you will get the entire data set every time, for events that have already happened this is fine since you only need to call the data once but for live events this would technically work but run poorly since the entire data set is replaced each time so any past data would also be overwritten which will lead to the site running very slow.

For live data the theory on the endpoints remains the same and as such the data returned is the same also but for live data you want it to update when new data is present not overwrite everything every second. This is where the MQTT/Websockets come in. This allows you to stream data from the API live and will only update when

there is new information to act upon, this is a far more efficient method than running the REST API for live sessions and will return a far smoother experience for the user.

2.3 Requirements

Overall I wanted this project to be a one stop shop for all data Formula One with the current events like standings and the current season while being able to see breakdowns from each session such as the session results and live replay and have the ability to go back through the years to see how sessions of the past went and replay those too!

Historical Data

For this historic side of the website I wanted the user to have the ability to select any of the seasons the API offers and be able to look through at the sessions and see what the results were, what the calendar looked like and be able to replay the sessions and see how the session unfolded. The users will be able to select any race of any year and view how each of the session went with a preview of all the sessions and a deeper look available at the full results in a more detailed tab. I also wanted the ability to look at the current standings of the championship table from the drivers standings and team standings with a preview on the home page showing the top 5 next to a card showing the previous race winner and a page showing the full driver and team standings.

A key focus for this section is scale-ability since each season has different number of races and different session names depending if it is a sprint weekend building this with scale in mind is key, as well as this building it in a way the CSS and HTML does most the heavy lifting and the JS simply just has to insert new elements is crucial otherwise the look of the site will vary page to page which is not what I wanted.

Live and Replay

Due to the way the API works these two options go hand in hand, I wanted the live and replay sections to share the same page and switch automatically depending on if the session is being broadcast live or if the user is watching a replay. I wanted this page to be the heart of the site as a way to show how the session went live, with a mini map of the location of all the drivers on the track as well as a timing tower to the left which would show the drivers position, interval gap to the next driver, gap to the leader and which tyre the driver is currently on.

A focus for this section was building a UI/UX that is clear and makes sense to the user knowing that information the section is outputting but also that is legible at a glance since the user is likely to be using this in conjunction with the current race session. Since each session also requires different data to paint a picture this should update depending on the session since gaps to the next driver are useless in a qualifying session when everyone is aiming for the top, likewise tyre life which is useless in qualifying. I would've loved to have a different tab instead of the mini map to show current sector and lap times to show the current speed of the drivers which can be interesting when a driver is on a different strategy and want to know the pace

difference. Finally, I also would've wanted another different tab replacing or complementing the mini map showing the current strategies which is particularly useful for mid race when drivers are trying something different and you want to know the different tyre lives the drivers have.

3 Design

Design was the very first place I started on this project since I had a vivid vision on what I wanted it to be but needed to realise it prior to making the code so I know I am happy with what I am working on and that the site has a clear design and direction.

3.1 Architecture

For this project I kept the architecture very simple and did not reinvent the wheel by creating external databases, though I will touch more upon that in the implementation section, I just wanted to make what was essentially a working prototype of the site. As such the architecture looks something like:

User → Web Interface → JavaScript → OpenF1 API → Data → UI.

The User will be spending their time interacting with the front end of the site via the HTML code in front of them to navigate, the plan was for the user to have to do very little and the site to do the heavy lifting with no confusion on how the site worked and how to use it effectively. As such the biggest thing I planned the user to interact with was the calendar and selecting which year they want to view.

This means the most of the site will be sitting in the JavaScript acting as a middle man processing what the user wants to view such as the year from the drop down on the front end they want to view. As well as this the JS needs to make request to OpenF1 and process the data retrieved and turn it into something useful and workable to put onto the UI, as the data stands it is intuitive but there are some quirks in the returned data such as in every endpoint the drivers are identified by their driver number not their name which was not something I thought about in the design phase initially and was learning a lot as I went on. The final key aspect the JS needs to handle is the replay functionality, since the replay cannot be streamed as if it is live this all needs to be done manually ordering the data via a time-stamp of when it came into the API and simulate time as if it is live and play the data back in order.

Since this was my first time working with an API in this capacity I wanted to keep things simple from the get go which meant I stuck to what I knew in working with HTML, CSS and JS but also work with the API directly. Once I got further into the project, I will go into greater detail later in the report, I realised a way to handle this data in a more efficient manner which was to set up an external database to act as an intermediary for the data which meant the historic data would update based upon timestamps I set-up but also the data request would be much faster since I wouldn't have to call the API every time. Once I realised this it was too late to make a meaningful change which is

why I consider this a working prototype of what I want from this site as it works but it is not very responsive due to this fact.

3.2 Data Flow

Since the architecture of the site is pretty straight forward so is the data flow, the skeleton is largely the same for every use case for the data from the timing tower to the mini map but they are all manipulated slightly differently to get the desired results for the function. The data flow is how the data processes through the website, how it starts its journey and how it ends. For this site the flow is:

API request -> API Response -> Processing -> UI

For historic data, at the top of the script is where all the API calls take place and return JSON responses. The responses are saved to constant variables since it doesn't change and is set in stone since the event has taken place, and once that is assigned the variables are sent to different methods to fill in the site and present data to the user. Processing for this is quite simple since the API response does most of it for you with the session result endpoint showing how a result ended, giving you each drivers position, whether they finished the race, DNF'd or DNS'd as well as a gap to the leader for the session. This can be combined with the driver end point to change the driver numbers for their names using the numbers as a key to retrieve the names and the interval endpoint to show gaps to the next car which could be useful for race sessions. This is then posted to the UI adding to the HTML code in the JS building a list of row elements.

For the replay functionality the core of the data flow is the same but takes a different shape, since the replays are mimicking a live broadcast the first thing to do is create an order for the data to flow correctly such as the location data for the mini-map. The endpoint for location gives an extremely large output since there is data for all the cars at every second of the session, the way to get around this is to create a time range for the data so you only retrieve data from the API for an amount for time, for example 2 minutes. This condenses the information down into an amount that is usable, the API request doesn't take too long/error due to the size and create a smooth experience. The rest of the data is smaller and thus can be called all at once but since the data has a time-stamp, we can make a snapshot at any moment when doing a replay to get a current state constructed by the replay engine and use the state to update the UI. The flow of data is the same, it just gets called in different ways due to the size of a response.

The UI itself does not output raw data from the OpenF1 API but rather processed data by the JavaScript to show the data needed for a specific use case allowing me to choose what data is shown and what is not.

3.3 UI Design

The design of the UI went through a few iterations until I decided on a direction to go in, I was using a wireframe tool called WebFlow which allows me to create a flow of the site and test it and see what works and what doesn't but most importantly how it looks.

The early designs for the site were going for a white minimal look since the colours of Formula One used to be mainly red and white since the 2018 re brand and wanted to for that look, but the more is developed it looked extremely generic and had no soul in the slightest. The design was flat and boring with the hero section not standing out enough and the content of the page being very underwhelming too. It wasn't all bad though, the flow of the site remained the same with the nav bar at the top as well as the order of the content following a hero section followed by content, even the way the content is formatted stayed the same. The biggest evolution of the design was the style after this version pivoted to a bolder, more colourful variation leaning on the bolder teams' colours contrasting with a darker background where the white was very hard on the eyes.

Old Design:

 Telemetry

[HOME](#) [CALENDAR](#) [SEASONS](#) [STANDINGS](#)



Australian Grand Prix

Albert Park Circuit | 22-24 March 2026

Winner
 George Russell

Lap
58/58

Fastest Lap
1:12.554

Race Results						
Race Finished 58/58 laps						
Pos	Driver	Team	Stops	Best Lap	Gap	Points
1	 George Russell	 Mercedes	1	1:12.554	Leader	25
2	 Kimi Antonelli	 Mercedes	1	1:12.334	+0.343	18
3	 Lewis Hamilton	 Ferrari	1	1:12.856	+3.565	15
4	 Charles Leclerc	 Ferrari	1	1:12.754	+4.565	12
5	 Max Verstappen	 Redbull	1	1:12.903	+10.434	10
6	 Isaac Hadjar	 Redbull	1	1:12.849	+11.434	8
7	 Lando Norris	 McLaren	1	1:12.687	+15.238	6
8	 Oscar Piastri	 McLaren	1	1:13.029	+18.937	4
9	 Pierre Gasly	 Alpine	1	1:12.554	Leader	25

Figure 1

Current Design:

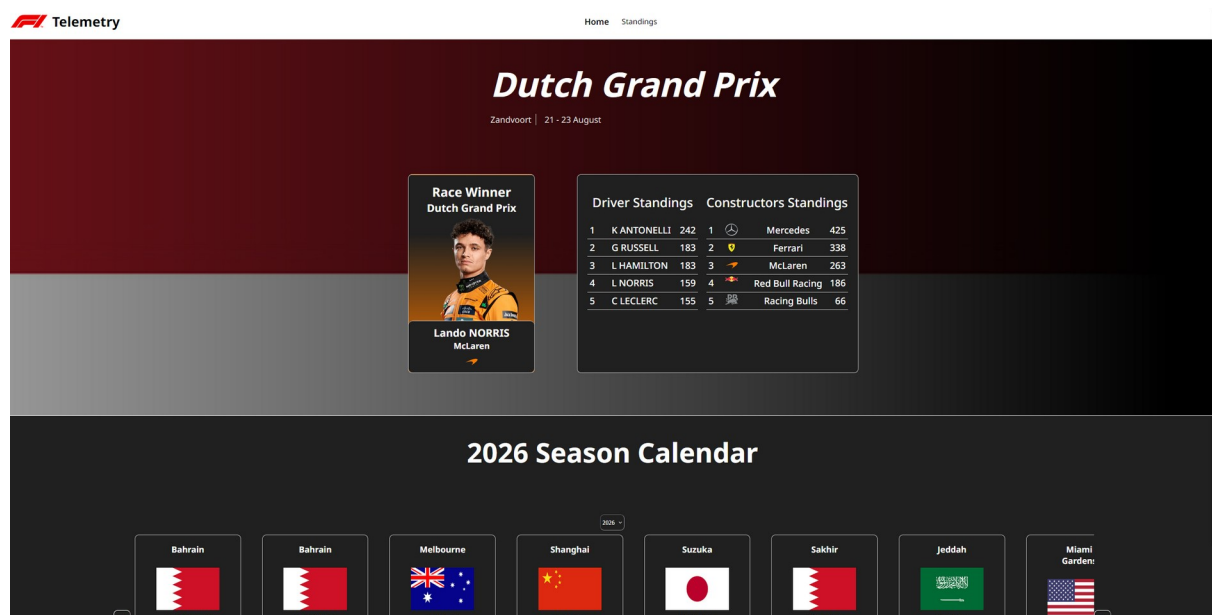


Figure 2

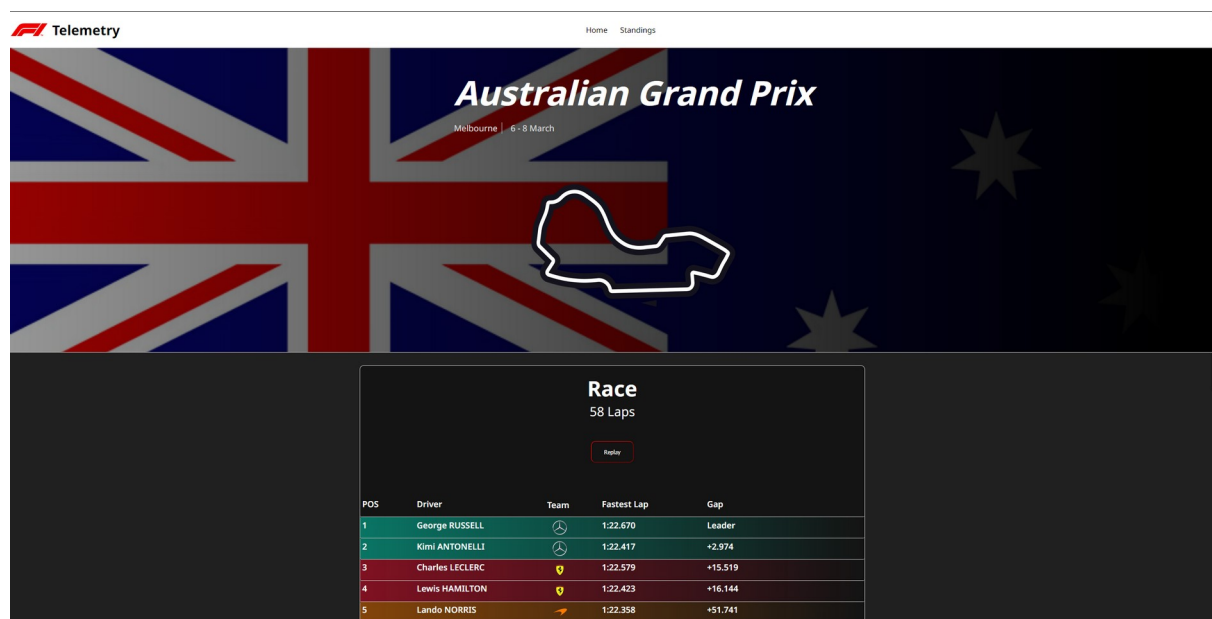


Figure 3

The navigation of the site is something I put a lot of thought into and think I have found a good balance. Starting from the home page you get the results of the previous race and the top 5 in the team and drivers standings, of which in the nav bar you can click standings to get the full view of both standings. Then on the home page there is the calendar with a drop down selecting which year you want to view, each card in the calendar has a button to view results which takes you to an overview showing every session with the top 5 and fastest lap and from there you can view the full results of each session. On each results page you can also view a replay of that given session. I

designed the navigation in a way you get more and more specific with a specific race weekend starting out with many options narrowing down to one session. I did consider adding a page where you can select any weekend and session from a drop down and reduce clicks but I feel this flow is nice and everyone that used it felt it was easy to use and makes sense.

For the standings of the site, this goes for the session results, standings or timing towers, the design ended up going through a lot of tweaks as the project progressed. Initially the rows for each team/driver were very bland trying as a remnant of the old white design of all the rows being the same colour with no team logo, only the position, driver name and any data after. But the site had a similar issue to the old design which was boring. The first thing to be added was the team logo to help differentiate each row rather than the rows being a wall of text, from there the natural progression is to add the colour in a gradient of the team colours. This breathed so much life into the side and so much colour I can no longer see the site without it, it helped so much with the glanceability of the site to see where each driver from each team stands in the replay and made such a big difference.

The timing tower went through a few iterations deciding which information is important to include for a race session, at the start there was no inclusion of interval gap only the gap to the leader and 2 columns related to pit stops, one for current compound and one for how many stops a driver has made. I decided to add the interval gap to the tower since late in a race the gaps become very big and if lapped the gap becomes "+1 Lap" which means you do not get a sense of the race behind the leader and who is catching who which made the viewing experience very stale but defeats the whole point of an insight application. There was an idea early on to make the information modular where the user can decide what info they want but the reality of the project set in and the added complexity this would bring was not feasible in the time I had and as such the idea was shelved, perhaps it would be integrated at a later date but in these 12 weeks it was not realistic working with a new API with new issues to solve.

The last big aspect to talk about is the mini-map for the replays. The design of this is very simple considering the fact it has to be drawn manually to work correctly which will be spoken about in the implementation section. The track is white to contrast against the background well and make it easily legible, as well as this the driver icons going around the track are in their team colours with drivers initials, there is no need for position as that is what the timing tower is for but knowing who is where when visually rather than juts numbers in the tower adds context to the gaps you see.

Showing where each driver is on track adds another layer to the site especially when you are rooting for a specific driver. Firstly, it gives the user a picture of a session, even for a practice session knowing where each driver is as a substitute to the broadcast adds a lot. Secondly for a race session, seeing where the drivers are on track is very important because it visualises the gaps you see on the timing tower but also as the race unfolds when lapped cars become a factor you can see if they are near one which means you can get into the race as a spectator. Finally, a lot of fun watching a race via software like this is getting into the data and speculate what will happen, when you can

see where the cars are on track you can try to guess what pit strategy is going to unfold and if they will get it right in the pit lane and if they emerge in traffic, it puts you into the shoes of the teams and how they will react to specific situations and see what they see.

4 Implementation

In this section of the report I will be discussing my implementation of the API into the website, how I handled issues relating to the code and any specific challenges/issues I faced during implementation and any unexpected issues.

4.1 API Integration

The first thing that I did was create a script that handled all the the API requests otherwise there would be a lot of repeated code which would be wasteful, this also means any error handling can happen site wide since it is all running via one method and make bug fixing that much easier too. Making an API request is reasonably simple since it is fetching the information via a URL which means all you need is a base URL since every request is sent through "https://api.openf1.org/v1" after this anything after is the endpoint whether it is "drivers", "race_control" or "session_result". Once you have established which endpoint this is where the parameters go to narrow down the response and get the desired information such as "session_key", "driver_number" or "gap_to_leader", this is outlined in the documentation on their site as to what variables an endpoint holds. An example of a URL to get information from OpenF1 is:

["https://api.openf1.org/v1/car_data?driver_number=55&session_key=9159&speed>=315"](https://api.openf1.org/v1/car_data?driver_number=55&session_key=9159&speed>=315)

When calling the API, which is where one issue reared its head, was the 2 common error types I was facing. The first one was a "404" error, this error was the easiest to work around since the only time it appeared was when there was a typo of some kind in the way the method was written. This error meant that the information asked for in the request could not be found. Since this meant the program couldn't continue anyway the way to combat this was have the method return empty. This showed in the UI as "UNDEFINED" and signalled something wasn't right to me and needed to be fixed. An example of this issue was very recently where for the Dutch Grand Prix, Red Bull driver Isaac Hadjar could not race due to injury meaning there was some movement in the drivers which mostly solved itself, but with the way the standings worked it called the drivers endpoint for the most recent event and drivers standings to format the information but since Hadjar did not race in the latest event but existed on the drivers standings there was an error since Hadjar's information could not be found, this issue was not entirely a 404 error but the 404 error helped expose a data integration issue assuming the information in the drivers standings endpoint would exist in the drivers endpoint. Here is the snippet of code dealing with the "404" error:

```
if (response.status === 404) return [];
```

The second kind of bug I faced, which didn't cause many issues but showed a flaw in how the site was built, was the "429" error. This error means the API is receiving too many calls at once, how I solved this was also a simple fix, it just meant when the API was overloaded any requests is failed to complete would retry until the request was completed including a cap on the amount of retries to stop is looping forever. Since this was not a content error this didn't affect the sites UI but the issue showed that the way I was making API request is not sustainable, and too frequent which made the performance of the site very slow and frustrating to use. The fix for this issue is not an easy fix since it is related to the way fundamentally, I have built the site. This is where the database idea came in but would require a full rebuild of the way data is fetched, the idea is to have a master database hosted on SQLite which held all of the information the API holds, the web page would call the information it needs from this database instead making the response time much faster since it is not pinging the API any more and there is a separate script that handles the updating of the database. Here is the code that relates to the "429" error handling:

```
if (response.status === 429 && attempt < retries) {  
    const retryAfter =  
        Number(response.headers.get("Retry-After")) || (at-  
tempt + 1)*2;  
  
    await new Promise(res => setTimeout(res, retryAfter *  
1000));  
    continue;  
}
```

A smaller workaround to help the performance of the site relating to the API is altering how information is gathered in the scripts. This didn't involve changing any of the related methods but rather changing the async variables, the way I was assigning information to variables was making an async method call to the API request scripts which is not the wrong way of dealing with it but it isn't the fastest since it has to wait for every API call to be made sequentially. One small change was making use of Promise.all, this meant I can make multiple API calls simultaneously increasing the speed of fetching the information drastically. There was some issues with this as other API calls need information from other calls especially when having to get the meeting key from the session endpoint to fill in the hero section of the site but this minor fix made things a lot faster where possible and reduce the time waiting.

4.2 Data Processing

When an API call is made the response is a JSON list of variables, for example if you asked the driver endpoint for a driver with a number of 63 it would return:

```
Driver{  
    Driver_number: "63",
```

```
    First_name: "George",  
    Last_name: "Russell",  
    Etc...  
}
```

This means when building the driver standings for example which has the drivers number, position and points you have to combine the driver endpoint and standings endpoint to produce an output that is easily readable to the user with the name included. This is where a lot of the complexity and API calls come from since for specific use cases the information needed is caught between 2 endpoints.

For the drivers standings as mentioned above I had to call data from 2 endpoints for one output but I got around it by creating a driver map which matched the driver numbers to the driver names meaning for the rest of the script I don't need to combine two endpoints and only need to do it once at the start of the script. Since the standings endpoint does sort the drivers for me, I don't need to manually do that so the only merging needed is the driver names.

For the session summary screen which displays all of the sessions on a given weekend there is multiple layers of processing going on. Firstly the page is given a meeting key which unlocks all the data in a given weekend and in this case I call the session API with the meeting key which returns all the session on a weekend in the response which acts as an array for example Practice 1 = response[0] meaning it is the first data block in the response. I can then use each of the sessions in that array and call the session results endpoint using each of the sessions and use each of their session keys and get the results for each session. The final method in this chain combines sessions, session results and the same driver map previously discussed to construct a session result for each session.

In both of these cases the UI was built incrementally while the data processing is taking place, there is a variable at the top of the method which starts empty and is build containing all the HTML elements needed to construct the element. There is a methods that builds it in which all needed data is passed into as well as a block of HTML code that was built prior as a text so I can build the styling into something I like before the whole site is populated and have to wing it with a messy page. The data accrued is then used to populate the HTML elements, save to a variable and the end of the method is updating the elements HTML with this new data. Here is a block of code outlining this process.

```
function fillDriverTable(standings, driverMap){  
    let rows = "";  
    let leader = null;  
    standings.forEach(driver => {  
        let driverInfo = driverMap.get(driver.driver_number);  
  
        if (!driverInfo) {  
            console.warn("No driver information found for:", driver);  
        }  
    });  
}
```

```

        return;
    }
    let driverName = driverInfo.full_name
    let teamName = driverInfo.team_name;
    let gap = leader === null ? "-" : "+" + (leader.points_current - driver.points_current);
    if(leader == null){leader = driver};

    rows += `
        <div class = "standing-row"
            style="background: linear-gradient(90deg, #${driverInfo.team_colour}60, transparent);">
            <h3 class = "pos">${driver.position_current}</h3>
            <img class = "standing-team-logo" src = "${getTeamLogo(teamName)}">
            <div class = "driver-headshot">
                <img width = "50" height = "50" src = "${driverInfo.headshot_url}">
                <h3 class = "driver">${driverName}</h3>
            </div>
            <h3 class = "points">${driver.points_current}</h3>
            <h3 class = "gap">${gap}</h3>
        </div>
    `;
    });
    drivers_table.innerHTML = rows;
}

```

4.3 Frontend

For navigation of the site there is not very much work that had to be done, I described the flow of the site above but the basic navigation was handled by the nav HTML element to change between the home page and standings page. Though the navigation looking deeper and deeper at each session the navigation is handled by JS since I need to pass the session and meeting keys in the URL bar this is how I add those and move the site along to maintain continuity between the pages to ensure the same event is being looked at, since scalability was a consideration I really did not want to manually make every page style as this is far from best practice, rather construct one page which fills in the information dynamically depending on the session/meeting being considered.

One element of the frontend I haven't touched upon a lot is the calendar on the home page which has a few elements worth touching upon. Firstly, the calendar runs on a track which is styled in a row which you can click through to view the whole calendar. This works similarly to the standings above where prior to the JS I handmade the HTML elements so I could text the implementation and use that as the base for the calendar

cards, each element is built up upon on the page initialisation. Each button left and right of the track allows the user to parse through the calendar to find the desired races they wish to look at, the amount to move left or right is calculated by getting the width of each card + the gap between each of the card elements on the track and then using the `viewport.scrollTo` method move by that amount * current index, for example it starts at 0 and moving right by 1 means you are viewing the 1 element.

As mentioned in the objectives one of the things I wanted to do is support mobile and pc devices, which turned out to be a lot more finicky than I expected since this isn't just changing the scale of the site. For example on the standings page the first thing I did was change the layout to support mobile, the hero bars do a great job at scaling to devices so this was not a concern but the biggest concern is the content. The standings page has the teams and driver standings side by side, for mobile due to the screen width if the screen is less than 800px width it switches to a column layout with the team standings below drivers. This was a more simple solution but this presented a new issue, since the scaling of the rows was absolute values (px) this meant when scaled down it was very crowded and not enough room for everything to fit, this is where a philosophy shift happened and from that moment I tried to factor in relative values such as 1fr or percentages to ensure scaling remains consistent at different scales. The home page by default does a good job at scaling, it is not perfect and there is some screen overhang but it is not the worst, I tried to scale all the pages as best I could but there is a few quirks left over I could not solve such as the point gap in the standings going over 2 rows but considering how it looked prior it is an upgrade. Below is the code for the standings page that deals with the scaling:

```
@media (max-width: 800px) {
  .all-standings {
    flex-direction: column;
    width: 100%;
    gap: 40px;
    padding: 15px;
  }

  .standings {
    width: 100%;
  }

  .standing-row {
    grid-template-columns: 20px 35px minmax(0, 1fr) auto 45px;
    column-gap: 6px;
    font-size: 15px;
  }

  .standing-team-logo {
    width: 50px;
    height: 30px;
  }
}
```

```
}
```

4.4 Historical Data

Historical data is implemented all through the project and makes it breathe as such I will go page by page focussing on elements that I have not focussed on yet. The first of which being on the home page with the calendar, as mentioned prior the key to the site is the calendar to be able to select any race to analyse and if you want to view previous seasons not the current one then there is a drop down just above it. The job of this is to create an easy way to change the seasons, before this the plan initially was to have a separate tab which can be seen in figure 1, and have the current calendar on the home page and the previous ones in another tab. But I felt this added click to the site for no good reason, as such I decided to localise it and have it all in one which is how it stands. The way this works is when the site loads it runs a method which checks if there are any years cached (will mention later) and if the current time is before the time stamp, if it is the drop down will populate with each available year. If there isn't any stored then it will get every meeting and sort them via year and add to a list every different year. Then I added an event listener to the drop down itself to listen if the value changes, if it does it run the method which populates the calendar with new data.

In the calendar section when the calendar is being assembled what is also happening is a meeting key is being assigned to the buttons on each card, this is very important to be done since this means all subsequent pages relating to the session will have a way to retrieve this information. Here is how this happens, since a meeting key is assigned to the button when it is click it will add the meeting key to the URL bar and change the page to the session summary. The session summary will contain the data from every session in a race weekend showing the top 5 results and the fastest lap of the session. This works via that meeting key in the URL bar, when the page loads that meeting key is retrieved via `URLSearchParams` method (see code below) JS enables me to use, when each of the session cards is built at the end the same thing as the calendar takes place but this time assigning the session key to each button since the next page is session results and when the button is clicked the results page loads and the session key is added to the URL bar where that is used to get all the session information relevant to the results page.

Once the session results page has retrieved the session key from the URL bar, it can be passed into the API function to get the data for the selected session. The returned data is then used and processed in line with the methods previously mentioned to work it into a palatable way for the user to digest. This also provides the session key to the replay button at the head of the page to pass through in the way way via the URL bar to the replay page and use the same methods to retrieve the information to make the replay run.

```
function getURLVariables() {  
    const query = window.location.search;  
    const params = new URLSearchParams(query);
```

```
    return params;
}
```

4.5 Replay System

The replay system was the most technically advanced part of this project since it required making a player from scratch and playback the data as if it was live. The first part of this is the data itself, the most important part of the data for this to work is a time stamp, otherwise the system cannot play it back sequentially accurately since the time stamp aligns all the data into chronological order showing what happened when which allows an accurate recreation of the data. The way the system works at first is my retrieving the data which is where the first issue arose, firstly the location data from the API contains data from every car, at every millisecond relating to their position on the circuit at any given time, the issue with this is the response is far too big and the response fails and does not work. The way I got around this issue is using the time to my advantage, first I needed a start point for the data which is the start of the session with a time offset, in this case I went with 1m 30s and that will get the location data within that time. When the data is reaching its end the API call is made again in another 1:30 chunk to keep the size of the API request the same but adding it on to the end of the data list to have the option to scrub through the data as you wish. All the other endpoint to get data from work slightly differently since the size of them is much smaller and can all be called at the start, the data is processed the same way except they are only called once at the start of the replay and not called again like location. But they still need the time element to run since the way the engine work relies entirely on time.

```
export class RestDataSource{
  async load (session_key, start, end){
    const [location] = await Promise.all([
      getLocation({session_key: session_key, "date>": start,
"date<": end}),
    ])
    return {location};
  }
  async load_non_repeated (session_key){
    const [position, laps, interval, race_control, stints] =
await Promise.all([
      getPosition({session_key: session_key}),
      getLaps({session_key: session_key}),
      getIntervals({session_key: session_key}),
      getRaceControl({session_key: session_key}),
      getStints({session_key: session_key})
    ])
    return {position, laps, interval, race_control, stints};
  }
}
```

```
}  
}
```

Once the data is processed then it makes its way through the replay engine, this is an object that is created in the Live.js script which contains all the necessary methods to make it run and implement things like play and pause in the future and most importantly the `GetStateAtTime()` method that is the component which produces the information needed to the replay to work. Firstly, `Start()` is called which is the play method essentially from the sense if you press the play button this is what runs which initialises the play method, this sets the `isPlaying` Boolean to true telling the engine to proceed with playing the data, fetches the data non repeated data and calls the initial chunk of location data to be called. Then `Play()` runs, this methods job is to calculate how much time has passed compared to the current time stamp and then multiplied this by the replay speed depending on how fast you want the replay to run, then the play method checks if more data is needed to be fetched comparing the current time to the last time data was called and if the set amount of time has elapsed to call `loadChunk()` and call the next 1:30 chunk of data and stops the replay progressing further than the amount of data in the last chunk if the replay goes beyond it. Then it creates the current state, when the replay engine is created an update method is added to it so when the engine runs `onUpdate()` this is what is runs to update the UI outside the engine using the state data, state data creates a snapshot of data at any given moment for example every driver will have a state which looks like:

```
DriverState[63]{  
    Location: X,Y,Z,  
    Interval: 0.543,  
    GapToLeader: 5.433,  
    Position: 3,  
    Stints: Soft 32 Laps  
}
```

This is true for all data such as `race_control` and `location` in a way that is easily accessible and this is important for in the future if I want to implement scrubbing as it keeps track of the data at a given moment in the data sense and how far through the data set retrieved that has progressed with the `scanIndex`, this is important as it allows the engine to know how far through each data set is and saves the place so when the user pauses it will resume from the same point rather than start again. The goal of this is to keep the data synced so that it doesn't progress at individual rates since different endpoints produce more data than others so if they progressed at the same time interval it will fall out of sync since some data doesn't need to be updated all the time like `race control` which can have an entry once every minute vs `location` which is constantly updated every millisecond.

The state is used when updating the UI showing the data at a moment, as mentioned before there is a method from Live.js passed into the engine which allows the engine to call that method from within which passes the state to which uses that data to update the UI in line with the data and give an accurate picture of what is going on.

4.6 Timing Tower

The timing tower in and of itself is a lot more simple since it runs entirely via the replay engine and the UpdateUI method. The methodology is also in line with the previous standings meaning each of the rows are constructed in HTML in the Live.js script and the information is added during this stage. Since there is updates every second this works the same way the information is simply changed each time to the next available section of data as per the replay engine state.

The one difference is since this is a tower showing position the tower has been sorted based on their positions so the top is first and last is at the bottom which is a difference in the data vs using session result since that comes pre sorted this does not. This is a stark difference to the using the session result endpoint during a live replay since all of the data is spread amongst a lot more endpoints with far more specialised information rather than one general endpoint producing most of the information, for example position, lap data, overtakes, pit information (how long a pit stop is), stint information (current tyre life) and race control are all separate and, in some cases, produce deeper information due to this specialisation.

Sorting code:

```
const sortedDrivers =
Object.entries(state.drivers).sort(([, a], [, b]) => a.position -
b.position);
```

Row construction code:

```
let pos = driver.position;
let driverInfo = driverMap.get(Number(driverNumber));
let logo = getTeamLogo(driverInfo.team_name);

let interval = formatIntervalGap(Number(driver.interval));
let leader = formatLeaderGap(Number(driver.gap_to_leader));
let compound = driver.stints?.compound?.slice(0, 1) ?? "-";
let tyreClass = "";

if (compound === "S") {
    tyreClass = "current-compound-soft";
}
```

```

        else if (compound === "M") {
            tyreClass = "current-compound-medium";
        }
        else if (compound === "H") {
            tyreClass = "current-compound-hard";
        }
        else if (compound === "I") {
            tyreClass = "current-compound-inter";
        }
        else if (compound === "W") {
            tyreClass = "current-compound-wet";
        }

        let tyreRow = `

```

One thing I meant to add but could not complete in time was changing the information displayed in the timing tower depending on the type of session which is important functionality. In a race session the data's focus will be more on how the drivers are interacting with each other in respect to the gaps to the leader and between themselves since the contest for points is related to position and thus who is moving up or down is very important. In a practice session or qualifying the most important information is mainly related to the gap to the leader in relation to their lap times, in practice the teams are mostly testing things which means who is making progress is important but also what they are doing regarding tyre strategy which can be seen in these sessions and what lap times were set on which tyre adding context to the lap times. In qualifying this is irrelevant unless it is a weather affected session in which it becomes important but most of the time the only important information is who is higher up the order in respect to their fastest lap time, in a dry session tyre strategy is

mostly the same as they are setting fastest lap times and thus on the fastest tyre compound.

4.7 Mini Map

Just like the timing tower the mini map runs off the replay engines state data but manipulates in a way that is very different to the timing tower. Firstly the UI needs to create the track but since it has 1m30s blocks sometimes it cannot fully complete the track image, this in theory could run on an SVG of the track map but the issue with this is becomes very hard to map the SVG image with all the x and y coordinates which means it is much easier to draw the track myself. Firstly all the state data is sent to the `drawTrack()` method which has the sole job of providing the visualisation of the map. The state is used to extract all of the location data and from there a list is created with every drivers state, then it determines who has the largest amount of location data at a given time since this is determined by the blocks set out prior which for a race is usually the driver starting first. Once this is done and it has the driver with the most data the location of that driver is then turned into a list of coordinates within the bounds outlined in the state to create a SVG path mapping out the circuit and finally joining all these points with a line which creates a visualisation of the track.

Since this visualisation is relying on the blocks of data this gets updated every time `UpdateUI` runs as long as the location list has increased in length therefore acting on new data, in hindsight I would have preferred to use this in line with the laps data getting location data from a particular lap (probably the fastest lap) that has the complete track data since the current implementation can only act on the 1m30s blocks the track doesn't complete immediately and in a practice or qualifying session can take a few laps to get a complete image. Another issue with this is even once the track is built it continues to update so if the driver with the most location data crashes the map, then updates with a random line following that driver's path.

Once the track is drawn then it needs to add the drivers onto the track to be able to follow them, this is done via the `updateMiniMap` method. Firstly the whole method is repeated for every driver on the grid therefore for every driver entry in state this will run unless the driver doesn't have any location data and is excluded so their dot doesn't appear. For each entry in drivers a dot is then created checking if a driver has one already if they do one is not created, this is done via an SVG group with different SVG element within it assigning each dot with a drivers number to keep it easily accessible to make changes if necessary. Now the dot is created the drivers information is then accessed to get their name and team colour with the name being added to the SVG group as a text element and assigned with the drivers name and changing the dot colour to the teams colour. Unlike the timing tower each drivers dot is not recreated every frame, once it exists it used `toMapCoordinates()` to find where it should be on track, since the track is drawn out of the moving cars iterating the cars via their location will keep it grounded on the track.

This works because the state provides it with the X and Y coordinates and the bounds is established in the replay engine which gives the maximum and minimum the coordinates that can be which are then normalised into a 1000x1000 space, one quirk of using an SVG is that the Y coordinate needs to be flipped since in SVG's the Y coordinate goes downwards by default I needed to flip the y coordinate to ensure the perspective of the track is correct.

```
function toMapCoordinates(x, y, bounds) {  
  
    const mapWidth = 1000;  
    const mapHeight = 1000;  
  
    const mapX =(x - bounds.minX) /(bounds.maxX - bounds.minX) *map-  
Width;  
  
    const mapY =mapHeight -((y - bounds.minY) /(bounds.maxY -  
bounds.minY) * mapHeight);  
    return {x: mapX, y: mapY};  
}
```

4.8 Performance

Performance of the website is not currently where I would like it to be and a big cause of this is the way I set-up how the site interacts with the API as mentioned previously. Every page is very slow to fetch information due to the fact each time it needs to get information it is making an API request every time and when it is dealing with vast amounts of data it slows down a lot. The replay page for example it has to gather information from the location, interval, stint, laps and race_control endpoint, I have mitigated this by splitting this information into information that needs updating in blocks and information that can be called all at once. Initially all of the data was called in those blocks as mentioned previously which caused instability when new data had to be called since there was a lot of data called and needed in a short space of time and because the program pauses if the time goes beyond the available data the experience was very choppy.

Another way I improved performance in the site is in the home page, I began to explore caching the data so that it can be called immediately which can be seen in the calendar and the calendar's drop down. They load very fast because it is saved locally, this works by saving all the relevant data in the local cache with a time stamp and a set period of time, in the calendars case that is six months since the calendar doesn't need frequent updates, when the page loads it checks the difference between the time-stamps and if it exceeds 6 months new data is fetched otherwise it loads the local data. The reason this could not be implemented site wide is because there is a finite amount

of local space that can be used for a web page and unfortunately the calendar alone exceeded this. This realisation gave birth to the idea to have an external database to hold the all the data and call from there as if it was local but this realisation came too late in development to implement leaving just a glimpse at what could have been.

Aside from the slow performance loading the pages the actual HTML runs very well, this is most noticed when switching from the home page to the standings, the site is very snappy and because the data in not as vast as other pages it loads very fast, almost like a snapshot of the vision I have and what I want this site to be.

5 Testing

This section will explain what tests I did during development of the site. I opted to test this project with an iterative mindset testing things as I went rather than with a big test suite at the end of the project, this allows me to catch issues when they arose not after when it is too late, This also allowed individual features to get a lot more testing since I would use them more frequently than once at the end.

5.1 Approach

As mentioned prior the testing approach was iterative rather than a big block at the end of development which was more natural to me to test features as I went and fix any issues at the moment with it being fresh. The main bit of testing I completed is functional testing, the main aspect of this kind of testing is asking questions about what is expected to happen and give it a pass or fail depending on the outcome. This is the main bit of testing for this site since the site is cultivating an experience so features working correctly is paramount in a good experience, especially when the user has little opportunities to break the system it is easier to build a more reliable system. Below is a table outline the functional testing that took place after bug fixing:

Test	Expected Result	Actual Result	Outcome
Does the hero section load as expected	The hero section is filled in with information relating to the previous race winner and the current top 5 in team and driver standings	The hero section successfully filled with all the correct information	PASS

Can the user select previous seasons and load the races from that season	The calendar begins on the 2026 season and via the drop down. menu the user can change the season and view the events from that year	The seasons began at 2026 but I could select any season with the calendar of that season from 2023 -2026	PASS
Can the user select a race weekend and view a glimpse of the results from each session	The user can select any race weekend from any of the seasons and view the preview from practice to the race with fastest lap of the session	The page loaded with the top 5 from each session outlining who was in what position, the session name and the fastest lap of the session	PASS
Can the user select a specific session from the chosen race weekend and see the full results	The user can view the full session results from the chosen session.	I selected the race from the 2026 Australian Grand Prix and saw the full results including the gap to the leader for all drivers	PASS
Can the user view a replay from the race	The user can view a replay from a given session including a timing tower and the mini map of the session	I can see a full replay of the 2026 Australian Grand Prix with a full timing tower showing the gaps to the driver ahead and to the leader as well as a mini map showing each drivers position on track.	PASS
Does the timing tower update as the race progresses and can you see each drivers position on track	The user can watch how the race unfolded showing where drivers are and their position	I can see each position in the mini map and the timing tower updating, although occasionally when there is no data for that moment for some drivers is switched to the "INTERVAL"/ "LEADER" default	Mostly
Does the viewport adapt to different screen sizes	When the layout goes below 800 px it should update to a more compact version	The layout does update to a smaller form factor for smaller devices but it is a bit broken 800-1500 providing a less than optimal experience. In ideal condition the experience is good	Partial
Does the API persist when the initial API request fails	Its should loop until there is a response.	Yes the API persist until there is data acquired and doesn't give up until it gets a response although this can take a	PASS

		while	
--	--	-------	--

Another method of testing I used was getting feedback from users, especially users that are also fans of the sport`and as such a good test to get feedback at any time. This usually consisted of me giving the testers a current build of the website and let them use the site as a user for a period of time which gave me a lot of understanding on how the general user will use and navigate the site. As a developer I will have designed the site with an intended way to interact with it in my mind but the average user will not use the site and abide by my made up rules, they will use the site how they see fit which is where the biggest issue came up which was related to the performance of the site. Most of the testers would not wait for the site to full load before pressing any buttons such as the replay, since the buttons data keys are initialised along with the bulk of the information if you interact with the site too fast you will end up on a blank page that will never load since it doesn't have the correct information as it was not initialised. Unfortunately this is not something I could fix purely because the keys needed are in the API responses for example the result preview that shows all the sessions each of those session keys are unique to the session they represent and the only way to get those keys is via calling all the sessions with the meeting key and assign them one by one. The biggest thing it gave me is perspective on how the average user will use the site and going forward how I can build it in a sustainable way that will make the users happy.

Finally the most important bit of testing I did was not conscience but during development I would just use the site as a fan of the sport, looking at data and being curious with this being very valuable since I found a couple edge case issues that would never have come up if I didn't use the site myself. The biggest one being the most recent one of Hadjar having to miss a race due to injury, due to the way I built standings page during the weekend after the sprint race I checked the standings only to find the standings page would not load. This is because I use the session result endpoint in conjunction with the driver endpoint to fill in each drivers name from the latest event, where this caused issue is because Hadjar did not compete in the latest event but did exist in the drivers standings there was an error. This showed me how important it is not only to have testers but also to be passionate about what you are working on at any moment and wanting to use it as a lot of the time many issues can be found by simply using the site myself.

5.2 Testing Results

The importance of the testing simply determining whether a feature worked, it is also about what you do with what you learned from testing those features and for me there were a few instances the testing influenced the outcome of the site.

The first example was caching data, through every stage of implementing the initial caching I was testing the site ensuring that the site behaved as expected and the same went the drop down in the calendar year selector. This was useful information when caching other aspects of the website such as the hero section or the standings testing those implementations during development to find it didn't work I knew the implementation was OK and needed to find another cause leading to discovering the local storage limit and that I had hit it. This demonstrated to me that local storage did not scale for big API responses which I did not initially realise when implementing, this led me to explore other ways to store data in a way that was scalable and fast with the most promising alternative was an external database to hold all of the API responses and call the database for the data which will scale for the future.

Another aspect that benefited from iterative testing is the mini map on the replay page, the early versions of the system called all the data in 10 second blocks which did work but did not provide a smooth experience since the API responses were not fast enough every time it required new information it would pause while fetching. This was not an implementation I was happy using and the pause can sometimes be very long if the API request was slow, so I altered the system with the first change being moving some of the repeated data to be called once at the start, the only API response that needed to be called incrementally is the location meaning that removing these four repeated endpoint calls improved performance significantly solving the stuttering issue prior.

The next change made was changing the interval at which data was called which was increased to the previously mentioned 1 minute 30 second interval, the biggest driving force behind this change was the drawing the track itself, since it ran on the most amount of location data when the chunk is 10 seconds there is not much data at all the track is barely complete, when the chunk is bigger the track is mostly complete which is still not ideal but it will get the data for the next lap and run as smooth as possible.

These examples demonstrate that the iterative testing approach was not limited to identifying bugs. It also influenced architectural and performance decisions throughout development.

6 Evaluation

6.1 Objective Evaluation

At the top of this report I set out 6 core objective from what I wanted this project to achieve creating a hub of information for Formula One fans and enthusiasts alike. However there were some challenges meaning not every objective was completely achieved due to limitations in my knowledge working with an API and the challenges during development as well as the time frame needed to realise my vision. I will be outlining each objective set out and evaluating whether it was achieved.

Information Hub

The first objective was to create an information hub for fans of the sport to get breakdowns of a race weekend which will show results from sessions but also general information about the sport such as drivers and teams standings. I feel this objective has been achieved

The flow of the site is inherently what I envisioned from the beginning where going deeper into the site will yield more specific information relating to the interests of the user giving the user the freedom on getting the information they want out of a site like this. As mentioned this philosophy for the site is held true holding a very particular flow from top to bottom in an intuitive way with the flow being:

Season → Meeting → Session → Result → replay.

The flow allows the user to start with a particular season, select a race weekend, select a specific session, view the session results and finally view a replay of the session giving the user the most detailed information of a given session and giving increasingly detailed information. The final replay provides the most detailed observation of a session showing exactly how the session turned out step by step. As well as this, the site also holds information relating to things outside individual session and adding the context in relation to the broader story of the season.

Overall I feel this objective has been met because the final application provides centralised information relating to the sport of Formula One. This being said I would have liked the information to go even deeper but due to time constraints this has not been possible but the initial objective still rings true.

Mini Map

The second objective stated that I would implement a mini map which displayed each drivers position on track in relation to one another which I feel is mostly achieved with the functionality but the performance of the implementation lets it down.

The primary functionality of the implementation is sound and works exactly as intended displaying a track map showing the current venue the user is currently spectating as well as each driver having their own dot move around the track with the drivers name within the dot and the dots styling being in their teams colours making it easy to find the driver you want to view on the map and where they are.

The reason I feel this is mostly achieved is unrelated to the functionality but rather related to the performance of the mini map. The way the system draws the track is inherently flawed, since it is designed to pull information in specific blocks this means the track doesn't finish drawing immediately and instead has to finish as the cars go round, the drivers location data influences the track being drawn this means that even after the track is drawn it will continue drawing and if the driver crashes it will add that data to the map giving the track irrelevant lines. Additionally to this with the way the track map is drawn it is most buggy in non race sessions, since it takes the location data from the driver with the most data if they don't drive in the session till later the track will not be drawn until then.

Timing Tower

The third objective states I will create a live timing tower to complement the mini map in the replay. The timing tower should display relevant information to the session such as position, gap to the leader, current tyre and lap time. This objective was mostly completed but with a couple features missing from the initial goal.

The timing tower is the other part of the live replay page which will show the current positions of the drivers with information that should change and update as the session progresses which was successfully implemented with minimal issues. The current implementation creates the timing tower live using the current data at a given time and reconstructs the tower every update which is efficient enough for the current use case and was reliable in testing.

The current implementation shows the drivers position, team, name, interval gap, leader gap and current tyre compound which does update, the reason why this is a mostly achieved objective is because the information does not update depending on the type of session is playing whether it is a practice session or race. This means when watching a replay of the practice session the information is not relevant to the session, data such as interval gaps are not super relevant to a practice like it is in a race, also the header does not change from laps which is a useless statistic outside of a race session since the other sessions rely on time remaining in the session.

Historic Results

The fourth objective for this project was to have the ability to go back through previous seasons and view the results of those sessions with a full replay available. This objective has been achieved completely through the drop down in the calendar on the home page.

This feature came very naturally to the project as it works off of the existing systems, when the meetings are updated it takes all of the years available and removed any repeated years which are then added to the drop down menu. The user can select any years that the API supports which is 2023 – now and will update going forward making the website essentially self sustaining providing the API does not change its endpoints in the near future.

The historical system also follows the flow of the website previously established in the report following the weekend getting deeper into the session to view the data the user wants to view. This made the feature significantly easier to implement since I built the other systems to be scalable with this in mind as as such when the calendar is put together the buttons are filled with the correct information to work as required.

Overall this feature is fully achieved with the historical results providing a system to view the events of the current season going back to 2023.

Mobile Compatibility

The fifth objective related to the compatibility across devices mainly related to mobile and pc devices since I wanted the site to work on everything. This was partly achieved since I did overlook this feature as something that would be easier than It ended up being.

The way it works is it switched the form factor when the width of the window is 800px or less which does work for certain devices and on the standings page changing the layout of the page place the team standings under the drivers standings rather than next to it.

The reason why this is a partially achieved is primarily because I did not spend the time testing the site at difference display sizes and did not test is often enough and run the site from that point of view. This meant that when running the site on my monitor and in developer mode changing the size of the window to mobile proportions most of the site works OK with some smaller bugs across the site pertaining to graphical issues however anything in between much like my laptop revealed more substantial issues with the implementation of the screen sizes. In the CSS of the project I tried to make it scalable with not using absolute measurements but in other aspects that I overlooked and didn't think to check it is lacking. The implementation provides reasonable support to most devices but with varying degrees of graphical issues depending on the screen size

Overall this feature works in intended use cases but is not implemented enough to be considered close to achieved anywhere near the standard I would be happy with due to not realising it was not compatible with edge case displays.

Race Strategy

The sixth objective was intended to be part of the live replay screen giving the user a look at how the race strategies were unfolding showing the life of the current set of

tyres and what previous compounds were used. This objective was not achieved with ground not being broken on this features development

The reason for development to have been missed is because I started this project with a vision of what I wanted for this project and as development progressed it became clear, despite my best efforts to manage scope, the scope of the project became unrealistic. Since this was my first time working with an API I underestimated the challenges that were presented and the time required to develop and flesh out the desired features meaning things had to be cut back which is a big loss to the site seeing as something like this would add that extra bit of depth to the replay when watching the session unfold.

Overall this feature would have been a great addition to the site with another layer of analysis as the race unfolded. However, prioritising the core historical data, mini map, timing tower and historic calendar meant that the project still served its original purpose just lacking some of those deeper layers.

6.2 The Positives

The biggest positive of the development of this site was the fact the core functionality was achieved as an information hub for Formula One fans providing a one stop shop for data on the current season and before. It also remains in line with the original vision I had set out for the site from the beginning keeping the design very similar to what the mock ups I created were as well as maintaining the flow I had planned to have from those designs progressing from the home calendar page → selecting a race weekend → selecting a session from the race weekend → view results from a specific session → view a replay of that session. The user also has no impact on the API calls since the site is built in a way designed that allows the users to consume the data on the site without changing the information it displays.

The replay system was my biggest technical achievement making a system that could play back the data from scratch keeping track of time passed rather than presenting static data with multiple endpoints being combined to produce a current state of a moment to be converted into a UI. The location data as mentioned before is chunked into 1m30s blocks as part of optimisation endeavours to keep the site running smoothly and on top of this using a scan index to keep all of the endpoint synced and pushing the data when it is ready and which bit of data should be staged next. This architecture leaves room for additional features such as pausing and scrubbing through a replay.

My iterative development approach lead to solving a lot of issues with the site in a timely manor rather than saving all of the bug fixing to the end which could end up breaking other aspects of the site opting to instead fix issues as I went along future

proofing many aspects of the site while in some cases influencing the architecture itself in regards to the external database idea.

The visual scheme of the site in my opinion looks very good providing a clean aesthetic in line with Formula One's colour scheme while maintaining a clear visual hierarchy between broad season information and specific race information. Each time drivers cards are shown in session results and timing towers have a distinct theme working off the same template but with varying styles in line with their team changing their teams logo and card colour in line with this producing a very colourful and bold scheme that is eye catching for the user.

The biggest positive of this project has been the realisation of my own limitations and forcing a feature into the site that would be under baked and unfinished before submission. When scope became unmanageable I didn't force myself into a crunch mode, I took a step back and changed priorities mid development to focus more on getting the core functionality complete before adding more features in a hurry while also recognising areas where I can move forward into the future to make improvements to the project.

6.3 The Negatives

The biggest negative of the project has to be the reliance on the OpenF1 API, since I did not consider the use of the API to be a limitation I built the site in a way that relies heavily on it with each script calling the information from scratch and despite a centralised way of making API calls I did not consider the fact it takes time to make those calls. With this implementation, the time it takes to make an API call can seriously hurt the users experience due to the time to fetch the information especially if the API returns a 429 error which can take a few loops to fix adding increased time to the loading of the site. As well as this large API calls can fail as I experienced with the location data for the replay system, I did implement a work around to this issue but it is a patch rather than a fix however there is a fix but would require a rebuild on how the site fetches its data. One fix to this issue would be an external database to store the information which would cut down on the time it takes to fetch the information. The current architecture requires API calls to happen repeatedly which increases the risk of a 429 error and as a result slowing the site down further so having this external database will help mitigate many of the issues the site currently has.

Another weakness of the site is the mini map and more specifically how the track map is drawn, in its current form it is reliant on the location data of the drivers but due to the limitation on the API this means the site is unable to fetch all of the information at once. This means that I have set up the location data to be fetched in 1m30s chunks which makes it run but at the cost of the track map being drawn incomplete, this

makes the site look poorly made and since it is drawing the track constantly if a driver crashes a line is drawn there permanently onto the track map. A better approach would be finding a particular lap from any driver and draw the track all at once based on that information.

One more negative related to the replay screen is the fact the timing tower does not change its information according to what session is currently being played, this means the same information for a practice session is used in qualifying and the race which is not relevant to those sessions. The system was made with the races in mind but I neglected the addition of the other sessions until it was too late to implement, this is true for the drivers positions as well as the header of the tower showing the lap for all sessions which should change to time remaining for other sessions that are not races. This was meant to be implemented but time for the project got away from me working on other aspects of the site meaning this one was left. The main reason for this was the scope management and keeping the site at a reasonable size meant that some aspects I wanted were not considered.

Speaking of scope management, as mentioned before this being my first project working with an external API there was a lot of learnings going on every step of the way, combining this with myself being a fan of Formula One I had so many ideas for what I wanted to include in this project which is evident from my mock up photos (*Appendix One*) from the start. This meant that despite scope being in the back of my mind the whole project scope crept up on me very fast resulting in features I intended to be featured to be cut such as the strategy screen on the replay, different information for sessions, Live sessions and more. In hindsight I would have designed the site in its absolute most basic form and prioritise items from essential to stretch earlier to keep expectations manageable.

6.4 Future Improvements

The first thing I would add in the future is the external database, I have mentioned many times that running directly off the OpenF1 API causes many issues relating to fetching information, mainly how long it takes to get information. This external database will help with many of these issues providing improved speed to the site along with improved scalability while reducing reliance on the OpenF1 API, if it stops working the site will have data to continue to function. This would be done with the use of SQLite to store the information with the biggest difference being able to add a time stamp to the data as a whole. In the current implementation the data mainly for the calendar updates once every 6 months, with the addition of a time stamp this means I can update the information (like the buttons on the calendar cards) in line with when the information has become available. This would add some complication in adding a

script to manage this data, update and fetch but in the long run would be a huge improvement.

Another improvement I would like to bring is to the replay system, the first change introducing controls for the system including play/pause, scrubbing and fast forward/backward. This would put more control in the users hands but also allowing the user to pick which part of the race they want to view, currently the system is a X5 speed mostly for testing purposes but being able to toggle that speed will be a big improvement for the replay-ability of each race to see what happened. Another improvement would be to fix the track generation and make the whole track appear at once, this is not a huge change but will significantly improve the quality of life of using the site meaning unexpected lines won't appear from drivers crashing producing a much cleaner experience for the user especially in non race sessions which are the most broken when it comes to track generation

One of the smaller changes I would implement would be making the timing tower update its information depending on the session type whether it is a practice or a race session. This is also a big quality of life update that would make the session more easy to read and get a better insight into what it going on with relevant information, for example interval gap to the next car is not very useful in a qualifying session when the goal is to climb as high as possible and close the gap to the leaders.

Finally the last changes I would love to make relate to missing features that didn't make the final build, the biggest of these being live session support. This would be able to use the same replay page that is used right now but would run from its own source running from the MQTT Web socket section of the API, the key difference compared to the REST API is the fact it does not call all of the data at once but rather update the information when there is something new and because it runs on the same endpoints as the REST API with the same data I can reuse the existing methods in Live.js with the only change being switching between live and replay functionality.

Another feature that didn't make the cut is the race strategy screen along with timing screens, this offers the user more places when watching live or a replay to get more insight into the session with the strategy being useful in a race showing tyre lives and what strategy is expected from that driver putting the user in the strategists seat guessing the calls for themselves. The timing screen would contain data from each sector and each drivers last lap time showing their current pace and where each driver is faster to everyone else at a given time, the main use case for this screen is in qualifying checking who is fast where and if someone is on a good lap at any time which can add excitement to the session you are watching.

7 Conclusion

In conclusion, this project set out to create an information hub for Formula One fans with historical data to view results from any race from 2023 to current with the results from any of those sessions including replays to view live style data breakdown including a timing tower and a mini map. The final implementation achieved its core purpose of the project, with the site providing a complete flow of data from selecting a season and meeting to session results and ultimately a replay.

The project stayed true to its initial objectives including the historical results, race weekends and replay previous sessions. The replay system was the most complex feature I have implemented in any project recently requiring data from multiple endpoints to be synchronised based on timing data and converted into usable data that was accessible to be posted to a UI. However multiple objectives were partially achieved with the mini map having specific limitations with track generation, the timing tower updating but not switching information between sessions and a responsive UI that doesn't adapt to smaller displays while planned race strategy screens could not make it into the project.

One of the biggest lessons I learned from this project was the importance of understanding limitations of external data sources especially when designing the architecture around them which was a big oversight which I learned about too late. The reliance of the OpenF1 API lead to performance and scalability issues that could have been avoided through an external database, the project also emphasised the importance of iterative testing and realistic scope management particularly when working with unfamiliar technology.

Overall, this project provides the functional groundwork to a Formula One platform that was originally envisioned. While it does not include all the features I had originally planned, the core functionality is complete and provides a strong foundation for future development. As touch upon there are many features that would benefit the web page and would allow this to grow into the definitive Formula One platform in the future.

Appendix

Appendix A:

Design Document Rationale – [Design Document.odt](#)

References:

OpenF1 (2026) *OpenF1 API: The open source API for Formula 1 data*. Available at: <https://openf1.org/> (Accessed: 1st September)

FastF1 (2026) *Introduction to FastF1*. Available at: <https://docs.fastf1.dev/> (Accessed: 1st September)

Team and driver logos, colours, and other Formula One branding used throughout this application are the property of their respective teams and Formula One Licensing B.V. They are used here for non-commercial, educational purposes only, as part of an independent MSc project. This project is not affiliated with, endorsed by, or sponsored by Formula One, the FIA, or any constructor team.

AI Disclosure

AI models such as ChatGPT and Claude were used in this project as a colleague to assist with ideas and structure in the report. It was not used to generate any text at all nor code. Everything I have completed in this project is my own.